

Aether Motors – Architekturdokumentation

Cloud-native Microservice-Webshop für Elektrofahrzeuge

Diese Dokumentation beschreibt die logische und physische Architektur des Aether Motors Webshops nach dem letzten Refactoring (Database-per-Service auch für Bestellungen, verteiltes Object Storage über MinIO, Wegfall lokal duplizierter Stub-Daten im SSR-Backend). Sie folgt der Vorlesungsstruktur: Architecture Component Descriptions, Architecture Decision Records, Deployment Decision Records.

Logical System Architecture

Aether Motors realisiert einen 3-Tier-Microservice-Webshop. Der Presentation Tier wird vollständig vom Web Shop Backend Server-seitig gerendert und übernimmt zusätzlich das Routing und Forwarding der Service-Aufrufe an die nachgelagerten Microservices. Der Business Logic Tier besteht aus sieben fachlich geschnittenen Services (Product, Cart, Order, Media, AI, Roadmap, Configurator). Der Data Tier umfasst zwei voneinander unabhängige MySQL-Instanzen (eine für den Produktkatalog, eine für Bestellungen), einen Redis-Cache für Warenkörbe sowie einen MinIO-Bucket für sämtliche Bildinhalte. Eine grafische Übersicht liefert das begleitende Foliendokument Architekturmodellierung.pptx.

Architecture Component Descriptions

ACD1: Web Shop Frontend (Browser)

Das Web Shop Frontend ist der vom Endkunden bediente Tier 1. Es besteht ausschließlich aus vom Web Shop Backend ausgeliefertem HTML, eingebettetem CSS sowie minimalen JavaScript-Inseln (z. B. Konfigurator-Interaktion, YouTube-Modal). Da kein Single-Page-Application-Framework aktiv ist, ist der Browser zustandslos – jeglicher Zustand liegt im Web Shop Backend bzw. in den Microservices. Lediglich die Cart-ID wird in einem httpOnly-Cookie persistiert, um zwischen Requests denselben Cart wiederzufinden. Bilder werden vom Browser direkt aus dem MinIO-Bucket geladen (anonyme Read-Zugriffe).

ACD2: Web Shop Backend (SSR + Service-Routing)

Das Web Shop Backend (Express + EJS, Port 3000) erfüllt zwei Aufgaben: (1) Server-Side Rendering aller Seiten (Home, Konfigurator, Galerie, Merchandise, Cart, Roadmap), inklusive eigener Geschäftslogik wie Form-Handling, Cart-Cookie-Verwaltung und Order-Flow; und (2) Routing- und Forwarding-Funktion: HTTP-Aufrufe unter /api/* und /configurator-ui/* werden an den jeweils passenden Microservice weitergeleitet (Product, Cart, Order, AI, Configurator). Bewusst ist das kein reines API-Gateway im klassischen Sinne (das nur Pass-through-Funktionen wie Routing, Authentifizierung und Rate-Limiting bereitstellen würde): das Backend besitzt eigene Templates, eigenen State und eigene Geschäftslogik. Es entspricht eher dem Backend-for-Frontend-Pattern,

das die Routing-Aufgabe mit übernimmt. Eingehende Requests werden ausschließlich von diesem Backend bedient; die Microservices sind nicht direkt öffentlich erreichbar. Daten zu Fahrzeugen und Merchandise zieht das Backend zur Laufzeit über HTTP/JSON vom Product Information Service (siehe ACD3); ein lokaler Cache mit 60 s TTL verhindert eine DB-Anfrage pro Render. Das Backend ist einem unpredictable Workload ausgesetzt und muss daher elastisch horizontal skalieren (siehe ADR5).

ACD3: Product Information Service

Der Product Information Service (Node.js, Port 3001) verwaltet sämtliche Stammdaten zu Fahrzeugen, Merchandise und allen Konfigurator-Optionen (Farben, Räder, Interieurs, Fahrwerke, Auspuffanlagen) inklusive der Verfügbarkeits-Joins pro Fahrzeug. Er stellt REST/JSON-Endpunkte unter /api/vehicles, /api/merchandise, /api/options und /api/search bereit und hält die Daten in der zugehörigen MySQL-Datenbank (siehe ACD9 und ADR7). Der Service ist – wie das Web Shop Backend – einem unpredictable Workload ausgesetzt und wird elastisch horizontal skaliert.

ACD4: Shopping Cart Service

Der Shopping Cart Service (Node.js, Port 3002) verwaltet kundenspezifische Warenkörbe. Jeder Cart wird als JSON-Dokument in Redis unter dem Schlüssel aether:cart:<cartId> abgelegt und bei jeder Mutation mit einer Sliding TTL neu geschrieben (Default 24 h, siehe ADR8). Da viele Kunden parallel einkaufen, ist der Workload unpredictable; der Service skaliert horizontal hinter einem Load Balancer und ist selbst zustandslos – der gesamte Zustand liegt in Redis (siehe ACD10).

ACD5: Order Service

Der Order Service (Node.js, Port 3003) übernimmt Vehicle- und Merchandise-Bestellungen aus dem Checkout, persistiert sie und stellt Bestellhistorie-Endpunkte bereit. Bestellungen werden in der eigenen MySQL-Instanz aether_orders abgelegt (siehe ACD12 und ADR11); JSON-Felder wie Customer, Address, Config und Items werden serialisiert in TEXT-Spalten gespeichert. Der Service ist zustandslos und kann beliebig horizontal skalieren; der gesamte transaktionale Zustand liegt in der Datenbank.

ACD6: Media Service

Der Media Service (Node.js, Port 3004) liefert Metadaten über die in MinIO abgelegten Bild-, Video- und 3D-Assets. Der Service streamt die Binärdaten selbst nicht aus, sondern verweist über öffentliche MinIO-URLs auf die Objekte (siehe ACD11 und ADR9). Dadurch bleibt die Antwortlatenz vorhersagbar und der Service skaliert praktisch beliebig, da er nur Metadaten ausliefert.

ACD7: AI Service

Der AI Service (Node.js, Port 3006) generiert auf Basis natürlich-sprachlicher Kundenwünsche eine passende Fahrzeugkonfiguration. Im Regelbetrieb wird Google Gemini (Modell konfigurierbar via GEMINI_MODEL) aufgerufen. Ist kein API-Key gesetzt, antwortet der Service mit HTTP 503 und reason gemini_api_key_missing; schlägt der Gemini-Aufruf selbst fehl, mit HTTP 502 und reason gemini_call_failed. Ist der Product Service nicht erreichbar, mit HTTP 503 und error Catalog unavailable. Es gibt bewusst keinen stillen Fallback

(siehe ADR10): Fehler werden ehrlich an den Aufrufer durchgereicht, der entscheidet, wie er sie dem Endnutzer kommuniziert.

ACD8: Roadmap Service

Der Roadmap Service (Node.js, Port 3007) liefert die Produkt-Roadmap (Meilensteine, Releases, Marketing-Phasen). Er ist rein lesend, hält die Daten in-memory und ist zustandslos – horizontal trivial skalierbar.

ACD9: Catalogue Database (MySQL, aether_motors)

Eine relationale MySQL-Datenbank (Container mysql, Host-Port 3306) hält sämtliche Produkt- und Optionsstammdaten. Schema und Seed-Daten werden über services/product-service/aether_motors.sql beim ersten Container-Start automatisch eingespielt. Die Datenbank ist gemäß Database-per-Service-Pattern (ADR7) ausschließlich dem Product Information Service zugänglich; alle anderen Komponenten greifen über dessen HTTP-API zu.

ACD10: Cart Cache (Redis)

Ein Redis-Container (Port 6379) persistiert die Warenkörbe des Cart Service. Persistenz erfolgt über Append-Only-File (--appendonly yes) auf einem Docker-Volume, sodass Cart-Daten einen Container-Neustart überleben. Die TTL pro Eintrag ist konfigurierbar (REDIS_TTL_SECONDS) und wird bei jeder Mutation refreshed; abgebrochene Sitzungen verschwinden so automatisch.

ACD11: Media Object Storage (MinIO)

MinIO ist die S3-kompatible Object-Storage-Komponente des Systems und beherbergt sämtliche Bild- und Mediendateien im Bucket aether-images. Der Bucket wird beim ersten docker-compose-Start automatisch durch den minio-init-Sidecar-Container angelegt und mit den im infrastructure/minio/seed-manifest.json hinterlegten Bildern befüllt. Anonyme Lese-Zugriffe sind erlaubt, sodass das SSR-Frontend die Objekte direkt referenzieren kann.

ACD12: Orders Database (MySQL, aether_orders)

Eine zweite, von der Katalog-DB getrennte MySQL-Instanz (Container mysql-orders, Host-Port 3307) hält ausschließlich Bestelldaten. Es existiert eine einzige Tabelle Orders mit skalaren Identifikations- und Statusspalten (OrderID, OrderRef, Status ENUM 'created' | 'paid' | 'cancelled', Type, CartReference, VehicleId, VehicleName, CreatedAt, UpdatedAt) sowie fünf JSON-Spalten (Config, Customer, Address, Items, Totals), die die Detail-Strukturen der Bestellung kompakt ablegen. Dieses Schema-Design wurde bewusst gewählt: Bestellungen sind in der aktuellen Ausbaustufe write-once und werden nicht über einzelne Items hinweg aggregiert; ein Star-Schema mit OrderItems/OrderEvents-Tabellen würde Mehraufwand ohne Mehrwert erzeugen. Das Schema wird beim ersten Start aus services/order-service/orders.sql geladen. Die Datenbank ist gemäß Database-per-Service-Pattern (ADR7/ADR11) ausschließlich dem Order Service zugänglich. Die physische Trennung von ACD10 erlaubt unabhängige Backups, unterschiedliche Skalierungsstrategien und vermeidet, dass Bestelltransaktionen den Katalog-Traffic beeinflussen.

ACD13: Configurator Service (Micro-Frontend)

Der Configurator Service (Node.js, Port 3008) ist ein Micro-Frontend für die Fahrzeugkonfiguration. Anders als die übrigen Microservices liefert er keine JSON-API für das SSR-Backend aus, sondern eine vollständige, eigenständige HTML-Seite (eigene EJS-Templates, eigenes Tailwind-Build, eigene Vanilla-JS-Insel) – inklusive Body-Shot des Autos, Farb-/Räder-/Interior-Selektoren, AI-Konfigurator-Panel und Checkout-Button. Diese Seite wird vom SSR-Backend an der Route /configurator über ein <iframe> eingebunden, dessen src auf /configurator-ui zeigt – das SSR-Backend proxied diesen Pfad (inkl. aller Asset-Requests) per http-proxy-middleware an den Service durch und setzt einen X-Forwarded-Prefix-Header, damit der Service korrekte Asset-URLs emittiert. Innerhalb des Service liegen drei Domain-Bausteine: (1) ein Product-Service-Client mit 60 s in-process TTL-Cache, (2) die Pricing-Engine mit Breakdown, (3) ein Image-Resolver, der aus (color, wheels) die passende vorgerenderte MinIO-URL zusammensetzt. Die Render-Route GET / liest die Auswahl aus dem Query-String, validiert sie gegen den live aus dem Product Service abgerufenen Options-Tree und rendert das EJS-Template. Klickt der Nutzer auf 'Continue to Checkout', schickt das iframe-JS per postMessage eine Nachricht an die Parent-Page; das SSR-Backend übernimmt von dort ab den klassischen Order-Flow. Damit bleibt die Cart-/Order-Verantwortung sauber im SSR-Backend, der Configurator-Service ist vollständig zustandslos (keine Datenbank, kein Cookie, keine Session). Bei Nichterreichbarkeit des Product Services liefert er eine ehrliche HTTP-503-Seite aus, konsistent mit ADR12. Die Begründung für diesen Schnitt steht in ADR13.

Architecture Decision Records

ADR1: Microservice-Architektur

Das System wird gemäß des Microservice-Architekturstils umgesetzt. Die einzelnen fachlichen Verantwortlichkeiten (Produktstammdaten, Warenkorb, Bestellungen, Medien, AI-Empfehlungen, Roadmap) lassen sich klar voneinander abgrenzen und besitzen jeweils starke interne Kohäsion. Microservices ermöglichen es, jede Komponente unabhängig zu entwickeln, zu deployen und zu skalieren.

ADR2: 3-Tier-Architektur (Presentation / Business Logic / Data)

Die Komponenten werden in drei Tiers gegliedert: Presentation Tier (Web Shop Backend mit SSR), Business Logic Tier (sechs Microservices) und Data Tier (zwei MySQL-Instanzen, Redis, MinIO). Diese Trennung erleichtert das Deployment (z. B. unterschiedliche Skalierungsstrategien je Tier) und die Absicherung (nur das Backend ist öffentlich erreichbar).

ADR3: Exklusives Server-Side UI Rendering im Web Shop Backend

Da der Webshop in der aktuellen Ausbaustufe nur eine vergleichsweise einfache, weitgehend statische Benutzeroberfläche bereitstellt und Inhalte nicht dynamisch nachgeladen werden müssen, wird ausschließlich Server-Side Rendering verwendet. Client-Side Rendering kann nachträglich für einzelne,

interaktive Bereiche (z. B. den Konfigurator) ergänzt werden, ohne die Gesamtarchitektur zu ändern.

ADR4: HTTP/JSON als Inter-Service-Kommunikation

Alle Microservices kommunizieren synchron über HTTP/JSON. Diese Entscheidung ist bewusst pragmatisch: HTTP ist im Web-Kontext ohnehin vorhanden, jeder Service kann mit den üblichen Sprachmitteln (fetch, axios) konsumiert werden, und Werkzeuge zur Observability sind weit verbreitet. Asynchrone Kommunikation über Message Broker wird in einer späteren Ausbaustufe in Erwägung gezogen, sobald langlaufende Hintergrund-Workloads hinzukommen.

ADR5: Elastic Horizontal Scaling und Load Balancer im Business Logic Tier

Die Komponenten Web Shop Backend, Product Information Service und Cart Service sind einem unpredictable Workload ausgesetzt und werden daher elastisch horizontal skaliert. Die Skalierung erfolgt im Zielbetrieb über einen PaaS-Hosting-Service des Cloud-Providers. Voraussetzung dafür ist, dass die Services zustandslos sind – im Cart Service wird dies durch externe Persistenz in Redis (ADR8) sichergestellt; im Order Service entsprechend durch Persistenz in einer eigenen MySQL-Instanz (ADR11).

ADR6: Relationale MySQL-Datenbank für strukturierte Stammdaten

Sowohl die Produktstammdaten als auch Bestellungen weisen klar relationale Strukturen mit Joins, Constraints und Bedarf an Filtern/Aggregationen auf. Für beide wird MySQL 8 eingesetzt. Document- oder Key-Value-Stores wurden verworfen, weil sie die geforderten Join-Operationen nur umständlich abbilden würden.

ADR7: Database per Service

Jede Microservice-Komponente, die Persistenz benötigt, besitzt eine eigene Datenbank, die ausschließlich von diesem Service zugegriffen wird. Datenflüsse zwischen Services laufen ausnahmslos über die jeweiligen HTTP-APIs. Dadurch bleiben Schemas unabhängig veränderbar; ein Schema-Update in Service A erfordert keinen Eingriff in Service B.

ADR8: Redis als verteilter Warenkorb-Cache mit Sliding-TTL

Warenkörbe sind kurzlebig, lesbar/schreibend hochfrequent und müssen für die Lastverteilung (ADR5) services-extern gespeichert werden. Eine relationale Datenbank ist dafür überdimensioniert. Redis bietet O(1)-Zugriff per Key, TTL-basierten automatischen Cleanup und Append-Only-Persistenz. Der Cart Service schreibt jeden Cart bei Änderung mit SET ... EX <ttl>, wodurch die TTL als Sliding Expiration wirkt: inaktive Carts verschwinden nach REDIS_TTL_SECONDS automatisch.

ADR9: MinIO als zentrales Object Storage für Bildmaterial

Für Bild- und Mediendateien wird MinIO – S3-API-kompatibles Object Storage – eingesetzt. Im Vergleich zu lokalen Static-Files unter public/ erlaubt MinIO kontrollierten Zugriff, unabhängiges Hosting und einfache Migration in produktive Cloud-Object-Stores (S3, GCS). Der minio-init-Sidecar lädt beim ersten Hochfahren des Stacks die im Manifest definierten Bilder einmalig in den Bucket. Anonyme Read-Zugriffe sind aktiviert, damit das SSR-Frontend Bilder direkt referenzieren kann.

ADR10: Ehrliche Fehlerweitergabe statt stiller Fallback im AI Service

Der AI Service ist von einem externen LLM (Google Gemini) abhängig. Eine frühere Implementierung hätte bei fehlendem API-Key oder Gemini-Ausfall mit einer deterministischen Default-Konfiguration geantwortet. Davon wurde bewusst abgesehen: ein solcher Stiller Fallback hätte dem Endnutzer eine echte AI-Antwort vorgegaukelt und dieselbe Klasse von Problemen produziert wie ein duplizierter Stub-Katalog im SSR-Backend (vgl. ADR12). Stattdessen liefert der Service strukturierte HTTP-Fehler aus: 503 mit reason `gemini_api_key_missing` wenn kein Key gesetzt ist, 502 mit reason `gemini_call_failed` wenn Gemini selbst ausfällt, 503 mit error `Catalog unavailable` wenn der Product Service nicht antwortet. Der Aufrufer (im Regelbetrieb der Configurator-Service hinter dem SSR-Backend) entscheidet, wie er den Fehler im UI darstellt (in unserem Fall: dezenter Status-Hinweis im AI-Panel, Selektoren bleiben funktional). Demos ohne API-Key funktionieren weiterhin – nur eben ohne AI-Vorschläge.

ADR11: Eigene Datenbank für den Order Service

Bestellungen werden in einer von der Katalog-DB getrennten MySQL-Instanz (`aether_orders`) persistiert. Damit gilt das Database-per-Service-Pattern (ADR7) konsequent auch für den Order Service. Vorteile: (a) Bestellungen können unabhängig vom Katalog gebackupt und skaliert werden, (b) Schreiblast aus dem Checkout-Prozess (Insert-lastig) beeinflusst die katalogseitigen Lese-Abfragen nicht, (c) datenschutzrelevante Bestellinformationen sind klar von den Stammdaten getrennt – wichtig für DSGVO-Audits.

ADR12: Kein duplizierter Produktkatalog im SSR-Backend

Das Web Shop Backend verwendet ausschließlich live-Daten des Product Information Service und hält keinen lokalen Stub bzw. JS-Fallback der Stammdaten mehr vor. Eine frühere Implementierung mit lokalen `data/vehicles.js` und `data/merchandise.js` wurde entfernt, da diese (a) DB- und Code-Daten auseinanderlaufen ließen, (b) ADR7 (Database-per-Service) im Geiste verletzen und (c) im Outage-Fall alte Preise an Kunden auslieferten. Stattdessen rendert das Backend bei Nichterreichbarkeit des Product Services eine ehrliche 503-Seite ("Aether is briefly offline"). Ein In-Process-Cache mit 60 s TTL fängt häufige Anfragen ab, sodass die DB nicht bei jedem Pageload getroffen wird.

ADR13: Konfigurator als Micro-Frontend hinter dem Backend-Proxy

Die Fahrzeugkonfiguration ist die fachlich wertvollste Domäne des Webshops und wird in einem eigenen, dedizierten Micro-Frontend umgesetzt (Configurator Service, Node.js, Port 3008). Anders als die übrigen Microservices, die ausschließlich JSON über HTTP an das SSR-Backend liefern, rendert dieser Service eine vollständige HTML-Seite (eigenes EJS, eigenes Tailwind-Build, eigene JS-Insel) und wird vom SSR-Backend per `<iframe>` eingebunden. Begründung: (a) klar abgrenzbarer Bounded Context – sowohl die Konfigurations-Logik (Pricing, Validierung, Image-Resolver) als auch die zugehörige UI gehören zusammen und bleiben in einem Service zusammen; (b) das SSR-Backend muss keine Konfigurations-Templates pflegen und keine Domänenlogik implementieren – es ist reiner Host; (c) unabhängiges Deployment – ein Konfigurator-Update erfordert keinen Deploy des SSR-Backends und umgekehrt; (d) horizontale Skalierung (ADR5) – der Konfigurator skaliert mit anderer Last-Charakteristik (vielen Selektor-Klicks pro Session) als das Page-Rendering. Konsistent mit ACD2 bleibt der Service nicht öffentlich erreichbar: das SSR-Backend proxied den Pfad `/configurator-ui` (inkl. aller Asset- und API-Subpfade) per `http-proxy-middleware` an den Service durch und setzt einen `X-Forwarded-Prefix-Header`. Der Service ist vollständig zustandslos – keine Datenbank, keine Session, kein Cookie. Die Source-of-Truth bleibt der Product Service, dessen Antworten der Konfigurator für maximal 60 s in-process cached. Die Cart-/Order-Verantwortung bleibt im SSR-Backend: beim Klick auf 'Continue to Checkout' sendet das iframe per `postMessage` eine Nachricht an die Parent-Page, die dann den klassischen POST `/configurator/order` Flow auslöst.

Deployment Decision Records

DDR1: Container-basiertes Deployment via Docker Compose (Entwicklung)

Für die Entwicklung wird das gesamte System über `docker-compose` orchestriert. Jeder Microservice, beide MySQL-Instanzen, Redis und MinIO laufen in eigenen Containern auf einem gemeinsamen Docker-Netzwerk; Adminer (über beide MySQLs konfiguriert) und die MinIO-Konsole bieten GUI-Zugriff auf die Datenbestände. Dieser Aufbau ist mit einem einzelnen Befehl reproduzierbar.

DDR2: Ein Container-Image pro Microservice

Jeder Microservice besitzt ein eigenes Dockerfile und wird zu einem separaten Container-Image gebaut. Eine `.dockerignore` stellt sicher, dass `node_modules`, Build-Artefakte und sensitive Dateien nicht in das Image wandern.

DDR3: Web Shop Backend auf AWS Elastic Beanstalk

Gemäß ADR5 muss das Web Shop Backend elastisch horizontal skaliert werden. Im produktiven Zielbild wird es auf AWS Elastic Beanstalk deployt – einem Fully-Managed PaaS, das automatisierte Skalierung, Health-Checks und Load-Balancing übernimmt.

DDR4: Microservices auf AWS ECS Fargate

Die fachlichen Microservices werden im Zielbetrieb in AWS ECS Fargate ausgeführt. Fargate liefert serverlose Container-Ausführung, eignet sich gut für die meist leichtgewichtigen Node-Services und erlaubt unterschiedliche Skalierungsprofile pro Service (Product und Cart elastisch, Roadmap in fixer Replica-Zahl).

DDR5: AWS RDS for MySQL für die Katalog-Datenbank

Die Katalog-Datenbank (ACD9) wird im Zielbetrieb als Managed-Service (AWS RDS for MySQL) bereitgestellt. RDS übernimmt Backups, Patching, Multi-AZ-Failover und Monitoring. Dies passt zu ADR6 und ADR7, ohne dass eine eigene Datenbank-Administration nötig wird.

DDR6: AWS ElastiCache for Redis für den Cart-Cache

Anstelle eines selbst betriebenen Redis-Containers wird im Zielbetrieb AWS ElastiCache for Redis genutzt. ElastiCache liefert horizontal skalierbare Redis-Cluster mit Failover, automatischen Snapshots und kompatibler Wire-Protocol-Implementierung – der Cart Service muss lediglich seine REDIS_URL anpassen.

DDR7: Object Storage – MinIO On-Premise, AWS S3 als Zielbild

In der Entwicklungs- und Demoumgebung läuft MinIO als Container und stellt die S3-API bereit. Da MinIO API-kompatibel zu AWS S3 ist, kann das System für den produktiven Betrieb ohne Anwendungs-Änderungen auf S3 umgestellt werden – betroffen sind ausschließlich die Umgebungsvariablen (MINIO_ENDPOINT/MINIO_PUBLIC_URL und Zugangsdaten).

DDR8: Geheimnisverwaltung über Secrets Manager

Sensible Werte (Datenbankzugänge für beide MySQL-Instanzen, MinIO-Credentials, GEMINI_API_KEY) werden im Code niemals fest hinterlegt. Lokal werden sie über docker-compose-Variablen gesetzt; im Zielbetrieb stammen sie aus AWS Secrets Manager und werden den Containern als Environment-Variablen injiziert.

DDR9: Zweite RDS-Instanz für die Orders-Datenbank

Die Orders-Datenbank (ACD12) wird im Zielbetrieb als eine weitere, getrennte AWS RDS for MySQL Instanz ausgerollt. Die physische Trennung von der Katalog-Instanz (DDR5) spiegelt die logische Trennung gemäß ADR11 wider und erlaubt: getrennte Backup-Pläne, eigene Wartungsfenster, differenzierte Storage- und Compute-Sizing-Profilen (Orders ist transaktional schreibintensiv, Katalog ist lese-lastig) sowie unabhängige Compliance-Audits.